

Cicli e condizionali

Control Flow Statements

Per **Control flow** (*flusso di controllo*) si intende:

un concetto fondamentale in programmazione, dove si verifica effettivamente l'esecuzione logica e pratica del codice, determinando come e quando le istruzioni vengono eseguite.

Senza di esso, un programma è semplicemente un elenco di istruzioni che vengono eseguite in sequenza. Con il flusso di controllo, è possibile eseguire determinati blocchi di codice in modo condizionale e/o ripetuto: questi blocchi di base possono essere combinati per creare programmi sorprendentemente sofisticati!

Nel contesto di Python, **il control flow** riguarda come gli statements (istruzioni) vengono eseguiti in base a condizioni o loop.

Gli statement di **Control Flow**, in Python, sono:

1. `if`, `elif`, `else` - Controllo condizionale:
 - Permette di eseguire blocchi di codice diversi in base a condizione booleane

≡ Esempio

```
x = 10
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

2. `for` loop - Ciclo con condizione
 - Itera su una sequenza o una collezione

≡ Esempio

```
for i in range(5):  
    print(i)
```

3. while loop - Ciclo con condizione

- Esegue un blocco di codice finché una condizione è vera

≡ Esempio

```
count = 0  
while count < 5:  
    print(count)  
    count += 1
```

4. break, continue, pass - Controllo del flusso all'interno dei loop

- break : interrompe un loop immediatamente
- continue : salta all'iterazione successiva
- pass : segnaposto che non fa nulla

≡ Esempio

```
for i in range(5):  
    if i == 3:  
        break # interrompe il loop quando i è 3  
    print(i)
```

5. try, except, finally - Gestione delle eccezioni

- Consente di gestire errori senza interrompere il programma

≡ Esempio

```
try:  
    x = int("abc")
```

```
except ValueError:  
    print("Conversion failed!")
```

6. `match` (Python 3.10+) - Pattern Matching

- Consente un controllo avanzato delle condizioni, simile a `switch` di altri linguaggi.

≡ Esempio

```
command = "start"  
match command:  
    case "start":  
        print("Starting...")  
    case "stop":  
        print("Stopping...")  
    case _:  
        print("Unknown command.")
```

✓ In sintesi

il *control flow* è ciò che permette di definire *come* il programma prende decisioni e ripete operazioni, rendendo il codice effettivamente utile e dinamico.

👁 Control Flow Statements

I **Control Flow Statements** sono una categoria più ampia che include **qualsiasi istruzione che controlla l'ordine di esecuzione** del codice. Questo comprende sia le istruzioni condizionali (`if` , `elif` , `else`), sia altri costrutti come:

- **Loop Statements:** `for` , `while`
- **Interruption Statements:** `break` , `continue` , `pass`
- **Exception Handling:** `try` , `except` , `finally`

- **Pattern Matching:** `match`

Detto ciò andiamo a vedere nel dettaglio i *conditional statements* (inclusi `if`, `elif` e `else`), *loop statements* (inclusi `for` e `while` e i relativi `break`, `continue` e `pass`).

Conditional Statements

Sono spesso indicato come dichiarazione `if-then`, permettono al programmatore di eseguire certi punti del codice che dipendono da alcune condizioni booleane.

Python ci mette a disposizione delle strutture per tradurre i diagrammi a blocchi.

Sintassi dei conditional statements

La sintassi è:

1. `if`:

esprime l'istruzione condizionale (significa "se" in inglese).

2. condizione: (es: `x<0`, etc.):

è la condizione che segue subito dopo l'istruzione `if`, se è `True` verrà eseguita l'istruzione indentata sulla riga successiva (se è presente), altrimenti non succederà nulla.

```
if condizione: #premere tab
    print (10)
```

🕒 Utilizzo delle parentesi tonde intorno alla condizione negli statements condizionali

non è necessario né obbligatorio usare le parentesi tonde intorno alla condizione negli statement condizionali, siccome la sintassi di python è progettata per essere pulita è leggibile le parentesi tonde sono omesse per convenzione.

☰ Esempio sintassi senza parentesi >

```
x = 10
>> if x > 0:
    print("Positive")
```

```
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

≡ Esempio sintassi con parentesi (non necessaria, ma valida) >

```
x = 10
if (x > 0):
    print("Positive")
```

Le parentesi non aggiungono nulla qui

Quindi le parentesi tonde per le condizioni in Python vengono usate in contesti specifici:

1. Raggruppare espressioni complesse per chiarezza:

```
x = 5
y = 4
z = 10
if (x > 0 and (y < 5 or z == 10)):
    print("Complex condition met")
```

2. Evitare errori di precedenza negli operatori:

```
a = 5
b = 1
c = 22
if (a + b) * c > 10:
    print("Condition met")
```

3. Evitare un'espressione su più righe:

```
x = 4
y = 3
z = 10
if (
```

```
x > 0
and y < 5
and z == 10
):
    print("Condition met")
```

✓ In sintesi

Per convenzione di linguaggio non si usano le parentesi tonde nelle condizioni semplici, tuttavia usarle migliora la chiarezza e la gestibilità delle espressioni più complesse benché non cambi nulla a livello di esecuzione del codice

Quindi, seguendo l'esempio sopra, la struttura dell'istruzione `if` è un'intestazione seguita da un corpo indentato.

Da notare come dopo la condizione vadano scritti i due punti (`:`) e vada aggiunto uno spazio bianco per denotare blocchi di codice separati.

Le istruzioni come queste vengono chiamate istruzioni composte.

Le istruzioni presenti nel corpo non hanno limite, posso metterne quante mi pare ma deve esserci almeno una istruzione.

🔗 L'istruzione `pass` ✓

Talvolta può servire che il corpo sia privo di istruzioni (di solito quando c'è del codice ancora da scrivere); in questo caso potete usare l'istruzione `pass`.

In python questo comando è una **istruzione nulla**:

Non fa assolutamente nulla e viene usata come segnaposto (placeholder) per mantenere la struttura sintattica valida, evitando errori.

```
(a < 3 or b > 10 and (c != 20))
```

Python ci mette a disposizione altre 2 conditional statements oltre l' `if` :

2. `elif` :

sta per else if ed esprime un'altra condizione dopo la condizione uno (cioè

l' if); quindi, se la condizione 1 è vera fai altro se non è vera fai quello che ti dice **elif**.

🕒 **Posso usare un numero imprecisato di elif dopo lo statement if**

```
if condizione: #premere tab
    print (10)
elif condizione 2 :
    print()
elif condizione 3:
elif condizione 4:
...
else
```

3. **else :**

viene utilizzato per eseguire un blocco di codice **solo se** tutte le condizioni precedenti (**if** e **elif**) risultano **false**.

Quindi:

```
x = -5
if (x > 0):
    print("Positivo")
elif (x == 0) :
    print ("Zero")
else:
    print("Negativo")
```

In questo caso questo blocco si esegue poiché x non è né > 0 né == 0.

Tant'è vero che questo statement **non ha bisogno di una condizione poiché viene eseguito solo quando tutte le condizioni precedenti sono false.**

Inoltre **deve sempre essere preceduto da un if o da un elif.**

Diverso se mettiamo **elif**.

```
if condizione: #premere tab
    print (10)
elif -10<a<0:
    print("a is true")
```

- La prima condizione (`if` condizione) viene verificata.
Se è vera, viene eseguito il blocco sotto di essa: il `print(10)` .
Se la condizione dell' `if` è falsa salta il `print` sotto l' `if` , e il flusso passa al successivo blocco `elif` (se presente).
La seconda condizione viene verificata solo quando la condizione dell' `if` è falso:
- 4. Se la condizione dell' `elif` è vera allora viene eseguito il blocco sotto l' `elif` ;
cioè `print("a is true")`
- 5. Se la condizione dell' `elif` non è vera il flusso va avanti, e passa al blocco `else` (se presente)
In questo caso non è presente nessun `else` perciò non verrà stampato nulla in output se entrambe le condizioni sono false

☰ Comportamento con `else`

Se a questo blocco di codice si aggiungesse l' `else` , il programma eseguirà l'azione definita sotto l' `else` solo quando tutte le condizioni precedenti (`if` e `elif`) sono false.

Es:

```
if condizione:
    print(10)
elif -10 < a < 0:
    print("Il valore di a è tra -10 e 0")
else:
    print("Nessuna condizione è soddisfatta")
```

Quindi se entrambi i blocchi `if` e `elif` sono falsi, il programma eseguirà il blocco sotto l' `else` cioè `print("Nessuna condizione è soddisfatta")` .
In conclusione possiamo dire che:

1. senza `else` il programma non esegue nulla se nessuna delle condizioni è vera
2. **Con** `else` , il programma esegue il blocco sotto l' `else` quando nessuna delle condizioni `if` o `elif` è vera.

`elif` e `else` da soli non hanno senso.

Bug

```
elif -10 < a < 0: print("Il valore di a è tra -10 e 0")
```

```
else: print("Nessuna condizione è soddisfatta")
```

Bisogna aggiungere che sia `elif` che `else` sono blocchi opzionali che si aggiungono allo statement `if`.

Annidare gli If

È possibile inserire un `if` dentro un altro `if`. Questa pratica viene chiamata annidamento(o nested if):

viene utilizzata per creare logiche condizionali più complesse, dove una condizione dipende dal risultato di un'altra.

In parole povere, puoi mettere un `if` dentro un altro `if` per eseguire un controllo più dettagliato solo se la condizione esterna è vera.

```
if cond1:
    if cond1:
        else: #riferito al if interno non esterno
    else: #riferito all'if esterno
```

② "Il modo per fare debugging in questo caso è usare il debugger" > Debugging?

si riferisce all'uso di strumenti di debug per analizzare l'esecuzione del codice e individuare eventuali errori o problemi.

In particolare, l'uso di un **debugger** aiuta a monitorare l'andamento del programma durante la sua esecuzione, in modo da capire perché una determinata condizione è vera o falsa e come il flusso del programma si sviluppa attraverso i vari blocchi `if`, `elif`, `else` annidati.

Cosa significa "debugging"?

Il debugging è il processo di identificazione, diagnostica e correzione degli errori nel codice.

Un **debugger** è uno strumento che permette di:

- **Esaminare il flusso di esecuzione di un programma riga per riga.**
- **Monitorare i valori delle variabili in tempo reale.**
- **Capire quale parte del codice sta causando un errore o un comportamento imprevisto.**

Relazione con il codice poco sopra

```
if cond1:
    if cond1:
        # codice
    else:
        # codice riferito al secondo if interno
else:
    # codice riferito al primo if esterno
```

Immagina che `cond1` sia una variabile booleana che può essere vera o falsa. Potresti non essere sicuro di come il flusso del programma stia seguendo questa logica a causa di un errore nelle condizioni. Il debugger ti permette di:

1. Eseguire il programma passo per passo:

Puoi eseguire il codice riga per riga per vedere se le condizioni `if` sono valutate correttamente.

2. Monitorare il valore delle variabili:

Durante l'esecuzione del programma, puoi osservare come il valore di `cond1` cambia e come viene trattato nelle varie condizioni annidate.

3. Verificare il flusso di esecuzione:

Puoi verificare se il flusso entra nel primo `if`, nel secondo `if`, o nell'`else`, e se le azioni vengono eseguite come previsto.

Per Ricapitolare

- **Analizzare il flusso di esecuzione** per capire quale parte del codice venga eseguita.
- **Controllare i valori delle variabili** (`cond1` in questo caso) per vedere se la condizione che ti aspetti venga soddisfatta.

- **Verificare l'annidamento:** assicurarti che l'`else` associato al secondo `if` venga eseguito solo se la condizione del secondo `if` è falsa e quella del primo `if` è vera.

Struttura di un if annidato

Per `if` annidato, come abbiamo già detto poco fa, è quando metti un blocco `if` all'interno del corpo di un altro `if`.

Ogni livello di indentazione in Python indica un livello di annidamento.

Per fare un esempio:

```
if cond1:
    if cond2:
        # codice eseguito se entrambe cond1 e cond2 sono vere
    else:
        # codice eseguito se cond1 è vera e cond2 è falsa
else:
    # codice eseguito se cond1 è falsa
```

L'importanza dell'indentazione

L'indentazione è fondamentale in Python perché indica il livello di annidamento del codice.

Se non viene rispettata l'indentazione corretta, Python restituirà un errore di sintassi. Ad esempio:

```
if cond1:
if cond2: # Questo non è corretto
    print("Entrambe le condizioni sono vere.")
```

In questo caso, Python si aspetta che il secondo `if` sia indentato sotto il primo `if`, per indicare che è un blocco annidato. Se non indenti correttamente, otterrai un errore.

☰ Per Ricapitolare

- Un `if` **annidato** è semplicemente un `if` dentro un altro `if`. Serve a creare condizioni più complesse, dove una dipende dal risultato dell'altra.

- L'indentazione è fondamentale per mantenere il flusso logico del programma.
- Ogni `else` si riferisce all'`if` immediatamente precedente al quale è associato.

Loops

I cicli (loops) in Python sono un modo per eseguire ripetutamente alcune istruzioni di codice, o un blocco specifico, finché una condizione risulti vera o per iterare su una sequenza di elementi.

Risultano fondamentali per automatizzare operazioni ripetitive.

For Loops

Fare i cicli in python.

Il `for` in python è utile quando abbiamo a che fare con le collections, in particolare quando dobbiamo scorrere liste o dizionari.

Sintassi

```
for + variabile in sequence(lista, dizionario, tupla):
```

Quindi:

- `for` :
parola chiave che indica l'inizio di un ciclo
- `variabile` :
una variabile temporanea che assume il valore di ciascun elemento della sequenza durante ogni iterazione del ciclo.
- `in` :
operatore che collega la variabile alla sequenza, indicando che si sta iterando attraverso di essa.
- `sequence` : qualsiasi oggetto iterabile, come:
 - lista : ["a", "b", "c"]
 - Dizionario: { "key1": "value1", "key2": "value2" }
 - Tupla: (1, 2, 3)

- **Stringa**: "ciao"
- **Range**: range(5)

Scorrere una lista:

```
lista_1 =[i for in range(10)]
for numero in lista_1 #numero è la variabile in cui va a finire
l'ennesimo valore all'interno della lista
print(numero)
```

In questo caso gli ho detto di stampare i numeri da 1 a 10, il `for` in python lo fa da solo, python prende il primo elemento al primo indice, il secondo al secondo e così via.

Così facendo ho la semplicità di non dover scorrere gli indici:

```
lista_2 =["a","b","c" ]
for lettera in lista_2
    print(lettera)
```

Il `for` legge `lista_2` e prende il primo elemento della lista e lo mette dentro la variabile `lettera`,

di conseguenza quando prendiamo la prima iterazione (cioè l'iterazione 0) stiamo stampando `a`.

Dopodiché, non avendo più istruzioni da eseguire e passa a `b`: ora l'iterazione 1 contiene `b` quindi quando faccio il `print` stampa `b`.

Dopo aver fatto ciò, non ho più istruzioni da eseguire e passo a `c`: ora `lettera` contiene `c`, quindi l'iterazione 2 contiene `c`.

Quando raggiunge la fine della lista esce dalla lista e il flusso continua con le altre righe di codice.

Annidare i for

Possiamo annidare i `for`:

```
lista_1 = [1,2,3]
lista_2 =["a","b","c" ]
for lettera in lista_2:
    for numero in lista_1:
        print(lettera,numero)
```

Per comprendere esattamente cosa fa un `for` e come processa le iterazioni, analizziamo passo dopo passo:

6. Inizio del ciclo:

- Il ciclo esterno inizia con la variabile `lettera` che assume il valore del primo elemento di `lista_2`, ovvero `"a"`.
- Il ciclo interno prende il primo elemento di `lista_1`, che è `1`, e stampa: `a,1`.

7. Iterazioni successive del ciclo interno:

- La variabile `lettera` rimane `"a"`, mentre il ciclo interno procede con il successivo elemento di `lista_1`, ovvero `2`. Stampa: `a,2`.
 - Il ciclo esterno ritorna alla variabile `lettera` che rimane `"a"`, e il ciclo interno prosegue con il successivo elemento di `lista_1`, ovvero `3`. Stampa: `a,3`.
- torno al `for` perché ho ancora elementi da numerare dentro la lista.

8. Fine del ciclo interno:

- Dopo aver iterato su tutti gli elementi di `lista_1`, il ciclo interno termina.

Non avendo da eseguire altri elementi il secondo `for` (`for numero in lista_1:`) torna al `for` principale (`for lettera in lista_2:`) che esegue `b` e si torna al secondo `for` che ricomincia la lista.

9. Ritorno al ciclo esterno:

- Il ciclo esterno avanza, assegnando a `lettera` il valore successivo di `lista_2`, ovvero `"b"`. Il ciclo interno si resetta e riparte con `1`.
- Stampa i valori: `b 1`, `b 2`, `b 3`.
- Finito quest'altro ciclo il secondo `for` torna un'altra volta al `for` principale che esegue l'ultima iterazione, cioè `c`, e si torna al secondo `for` che ricomincia la lista.

10. Ultima iterazione del ciclo esterno:

- La variabile `lettera` assume l'ultimo valore di `lista_2`, ovvero `"c"`. Il ciclo interno riparte nuovamente, iterando su tutti gli elementi di `lista_1`.
- Stampa i valori: `c 1`, `c 2`, `c 3`.

Una volta finite le liste continua a lavorare altro codice

In totale vengono printate 9 righe, cioè 3 elementi in `lista_1` moltiplicati per 3 elementi in `lista_2` ($3^2=9$).

L'output è:

```
a 1
a 2
a 3
b 1
b 2
b 3
c 1
c 2
c 3
```

Nota

Ogni iterazione del ciclo interno si completa completamente prima che il ciclo esterno avanzi al successivo elemento.

Questo comportamento è fondamentale per comprendere l'esecuzione dei cicli annidati in Python.

Immaginiamo che tutto questo sia una matrice:

Iterazione	indici	Colonna 1	Colonna 2	Colonna 3
prima iterazione	0	a1	a2	a3
seconda iterazione	1	b1	b2	b3
terza iterazione	2	c1	c2	c3

Mettiamo caso ho una lista di liste e voglio estrarre gli elementi singoli all'interno, ad esempio `b2`.

Io posso accedere ai valori di una lista di liste tramite indici, per fare ciò:

Dichiarare la lista `L`:

```
L=[[a1], [a,2], [a,3]],
  [(b,1), (b,2), (b,3)];
  [(c,1), (c,2), (c,3)]
#Questa lista di liste che contiene tuple fa riferimento alla
tabella di matrice che si trova poco sopra
```

```
l[1][1] #Accede all'elemento b2
```

possiamo anche usare anche i cicli `for` per accedere agli elementi:

```
for i in range: (len(lista_2))  
#for i che varia nel range che va d 0 fino alla lunghezza della  
lista 2  
    for j in range:(len(lista_1))  
        #j varia nel range da 0 fino alla lunghezza della lista_1  
        print (L[i][j])
```

Alla prima iterazione:

- `i = 0` , `j = 0` → stampa `a1`
- `i = 0` , `j = 1` → stampa `a2`
- `i = 0` , `j = 2` → stampa `a3`

Dopo aver terminato il ciclo interno:

- Torniamo al ciclo esterno: `i = 1`
- Il ciclo interno si resetta:
 - `j = 0` → stampa `b1`
 - `j = 1` → stampa `b2`
 - `j = 2` → stampa `b3`

Infine:

- Torniamo al ciclo esterno: `i = 2`
- Il ciclo interno si resetta di nuovo:
 - `j = 0` → stampa `c1`
 - `j = 1` → stampa `c2`
 - `j = 2` → stampa `c3`

Nota

Si consiglia di usare lettere singole quando si a che fare con indici (la convenzione prevede i e j)

Collegamento con l'esempio iniziale

`i in range` va da 0 a 3 stessa cosa per `j in range` .

Se io ho una lista del genere e voglio accedere a tutti gli elementi, posso usare gli indici come visto sopra, io uso allora il `for` perché io voglio crearmi le coordinate (`i` e `j`) per prendermi tutti gli elementi, cioè sto facendo una enumerazione:

cioè generare, in questo caso, tutte le combinazioni della matrice .

```
L = [{"a", 1}, {"a", 2}, {"a", 3},  
      {"b", 1}, {"b", 2}, {"b", 3},  
      {"c", 1}, {"c", 2}, {"c", 3}]
```

Per prendere tutti gli elementi al suo interno uso il `for` :

```
for i in range(3):  
>   for j in range(3):
```

Abbiamo messo come argomento dei due `in range()` `3,3` perché so che questa lista contiene 3 sotto-liste con 3 elementi.

la funzione `range` :

ritorna un range di una lista di interi che va da 0 a 2 che sono esattamente gli indici .

Adesso stampiamo quanto fatto finora

```
`for i in range(3):`  
    for j in range(3):  
print(L[i][j])
```

prima iterazione `i=0 j=0` quindi `a,1`

seconda iterazione `i=0 j=1` quindi `a,2`

terza iterazione del ciclo interno `i=0 j=2` quindi `a,3`

Torno al `for` principale(`for j in range(3)`)

`i=1`

torno al secondo `for` (`for j in range(3)`)

`j=0` quindi si resetta e si ri-esegue da capo

prima iterazione `i=1 j=0` : `b,1`

seconda iterazione `i=1 j=1` : `b,2`

terza iterazione `i=1 j=2` : `b,3`

torno al `for` principale

`i=2`

torno al `for` secondario che si resetta

Prima iterazione `i=2 j=0` : c,1

seconda iterazione `i=2 j=1` : c,2

terza iterazione `i=2 j=2` : c,3

Quindi l'output stampato sarà:

```
("a", 1)
("a", 2)
("a", 3)
("b", 1)
("b", 2)
("b", 3)
("c", 1)
("c", 2)
("c", 3)
```

Il ciclo `for` crea le coordinate (`i` , `j`) necessarie per accedere a tutti gli elementi, generando tutte le combinazioni possibili degli indici.

Questo approccio è utile per scansionare matrici o strutture annidate in Python.

Filtrare elementi specifici con cicli annidati

Supponiamo di voler stampare solo gli elementi delle tuple che contengono numeri pari:

```
for i in range(3):
    for j in range(3):
        if(L[i][j][1]%2)==0: #controlla se il numero è pari
            print(L[i][j])
```

Spiegazione

11. **Condizione** `L[i][j][1] % 2 == 0` :

- `L[i][j]` accede alla tupla nella posizione `i`, `j` della matrice.
- `[1]` accede al secondo elemento della tupla (il numero accanto alla lettera).
- Il modulo `% 2` verifica se il numero è divisibile per 2 (pari).

12. **Esempio di iterazione:**

- Alla prima iterazione, $i = 0$, $j = 0$, la tupla è `("a", 1)`. Il numero 1 non è pari, quindi non viene stampato.
- Alla seconda iterazione, $i = 0$, $j = 1$, la tupla è `("a", 2)`. Il numero 2 è pari, quindi viene stampato.
- Questo processo si ripete, stampando solo le tuple con numeri pari: `("a", 2)`, `("b", 2)`, `("c", 2)`.

L'output quindi sarà:

```
("a", 2)
("b", 2)
("c", 2)
```

Filtrare numeri pari in una singola lista

Per stampare solo i numeri pari in una lista:

Soluzione con indice

```
lista_1 = [1,2,3]
for num in range(len(lista_1)):
    if lista_1[num]/2==0:
        print(lista_1[num])
```

Soluzione più semplice

```
for num in lista_1:
    if num /2==0:
        print(num)
```

La seconda soluzione è più concisa e preferibile quando non è necessario accedere agli indici direttamente.

In entrambi i casi, l'output sarà:

```
2
```

Attenzione!

Se usiamo la funzione range mi restituisce solo gli indici

La funzione range

Restituisce una sequenza di numeri.

Particolarmente utile quando vuoi ripetere un'operazione un numero specifico di volte o iterare su una sequenza numerica.

```
range[3] #restituisce una lista che va da 0 a 2, cioè va dall'indice 0 all'indice 2
```

Possiamo specificare i parametri di inizio (start), fine (end) e passo (step)

```
range[10] #  
range[5,10] #quindi va dal quinto indice al nono  
range[1(start),10(end), 2(steps)] #che fa? Stampa tutti i dispari e saltando un solo numero  
#possiamo far variare i numeri come ci pare
```

Nel primo caso(range[10]):

stampa tutti gli indici della lista

Nel secondo caso(range[5,10]):

va dal quinto indice al nono indice

Nel terzo caso(range[1(start),10(end), 2(steps)]):

Genera 1, 3, 5, 7, 9

 **Tips & Tricks: Usare range() con len() iterare su una lista con indici**

```
frutti = ["mela", "banana", "ciliegia"]  
for i in range(len(frutti)):  
    print(f"L'indice {i} contiene {frutti[i]}")
```

Output:

```
L'indice 0 contiene mela  
L'indice 1 contiene banana  
L'indice 2 contiene ciliegia
```

🕒 Curioso notare che il significato degli argomenti di `range` è molto simile alla sintassi di `slicing` che abbiamo trattato nella lezione sulle Collections.

List comprehension

La **list comprehension** è una sintassi compatta e più efficiente di Python per creare liste:

invece di usare un ciclo `for` tradizionale per aggiungere elementi a una lista, la list comprehension ti permette di fare tutto in una sola riga di codice, rendendo il processo più conciso e leggibile.

Sintassi

La sintassi di base della list comprehension è:

```
[espressione for elemento in iterabile]
```

- `espressione` :
è l'operazione o il valore che vuoi inserire nella lista.
- `elemento` :
è ogni singolo elemento dell'iterabile (come una lista o un range) su cui stai iterando.
- `iterabile` :
è l'oggetto su cui iteri (ad esempio, un `range`, una lista, un dizionario, ecc.).

Creazione di una lista con valori interi ordinati

Abbiamo bisogno di creare una lista con un insieme di interi ordinati:

```
lista_1=[i for i in range(10)]  
print(lista_1)
```

L'output sarà:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Creazione rapida di dizionari

Si posso applicare pure per creare i dizionari:

```
dict_1={i: i for i in range(10)}  
print(dict_1)
```

L'output sarà:

```
{0: 0, 1: 1, 2: 2, 3: 3, 4: 4, 5: 5, 6: 6, 7: 7, 8: 8, 9: 9}
```

Esempio con passo specifico nel range

Anche qui, come per il range, possiamo specificare un passo:

```
lista_1={i: i for i in range (0,10,2)}  
print(lista_2)
```

L'output sarà:

```
[0, 2, 4, 6, 8]
```

☰ Per Ricapitolare le caratteristiche della List Comprehension



1. Funziona sia su liste ordinate che disordinate.
2. È utile per semplificare la creazione di liste o dizionari.

Esempio: creazione di una lista con un ciclo tradizionale

Vediamo ora come creare una lista con un ciclo for tradizionale:

```
length = 5  
numbers = []  
for i in range(1, length + 1): # Range parte da 1 e include  
    length (grazie a `+1`)  
    numbers.append(i)  
print(numbers) # Output: [1, 2, 3, 4, 5]
```

Spiegazione passo per passo:

- **Inizializzazione:** La lista `numbers` è vuota all'inizio.

- **Iterazione:**

- Al primo ciclo, `i = 1` . Viene aggiunto `1` alla lista, che diventa `[1]` .
- Al secondo ciclo, `i = 2` . Ora la lista diventa `[1, 2]` .
- Questo processo si ripete fino a che `i` raggiunge `length + 1` .
- **Risultato finale:** La lista `numbers` contiene tutti i numeri da `1` a `5` .



Voglio iterare sugli indici della lista

```
l = ['a', 5, '?', True, 5]
range(len(l))
range(0,5)
[x for x in range (len(l))]
for i in range (len(l)):
    print(i)

0
1
2
3
4
```

qui faccio

```
for i in range (len(l)):
    print(i)

a
5
?
True
5

for i in range (len(l)):
    print (f"indice:{i} - valore{l[i]}")

indice : 0 - valore : a
indice :1 - valore :5
indice: 2 - valore : ?
```

```
Indice : 3 - Valore : True
Indice : 4 - Valore : 5
```

Qui invece:

```
\>\>\>l
['a', 5, '?', True, 5]
\>\>\> for ind in range (len(l)):
    el= l[ind]
    print(el* ind)

5
??
3
20

\>\>\>l
['a', 5, 'formaggio', True, 5]
\>\>\> for ind in range (len(l)):
    el = l[ind]
    print(f"{el} * {ind} = {el * ind}")

a*0 =
5*1 = 5
formaggio *2 = formaggioformaggio
True* 3 = 3
5\*4=20
```

I dizionari

si dichiarano con le parentesi graffe e spno un insieme ordinato di chiavi-valori

```
d = {0: "Andrea", 1:"Dioni", "Marco": True}
d[1]
\>\>\>d[" Andrea"]
d[0] = 'Gabriele'
\>\>\> {0: 'Gabriele', 1:'Dioni', 'Marco':True}
d[2] = 'Luca'
```



```
d.pop(2)
'Luca'
```

Per accedere a tutte le chiavi di un dizionario usare la funzione `.keys()`

```
d.keys()
dict_keys([0, 'Marco', 1])
```

la funzione `.values()` :

```
d.values()
dict_values([Gabriele, True, 'Benedetta', 56, True])
```

Per iterare sui valori di un dizionario

```
for value in d.values():
    print(value)

Gabriele
True
Benedetta
56
True

for key in d:
    print (f"Chiave: {key} - Valore{d[key]}")

Chiave : 0 - Valore Gabriele
Chiave : Marco - Valore True
Chiave : 1 - Valore Benedetta
Chiave : Aldo - Valore 56
Chiave : 3 - Valore True
```

While Loops

Il ciclo `while` esegue un blocco di codice fintanto che una condizione specificata è vera.

Quindi in altre parole **è usato per ripetere un blocco di codice** fintanto che una condizione è vera

Sintassi

Per dichiarare un `while` devo scrivere:

```
while condizione:  
    #blocco di codice da eseguire
```

≡ Esempio

```
x = int(input("Inserisci un numero: "))  
while x <= 5:  
>     print(x)  
>     x += 1
```

- L'utente inserisce un valore per `x`
- Il ciclo continua fintanto che `x` è minore o uguale a 5 .
- Ad ogni iterazione, `x` viene incrementato di 1 e il valore di `x` viene stampato .

Per cui se `x=3` , l'output sarà:

```
3  
4  
5
```

In questo caso se si inserisce un numero maggiore di 5 , ad esempio 6 , l'output non sarà stampato.

Break e Continue nei cicli

Queste due parole chiave modificano il comportamento dei cicli.

13. Break :

Interrompe il ciclo in corso, terminandolo immediatamente.

≡ Esempio Pratico

```
for i in range (10):  
    if i == 2:  
        break  
    print(i)
```

L'output sarà

```
0  
1
```

In questo caso il ciclo si ferma quando `i` è uguale a 2.

14. Continue :

Salta l'iterazione corrente e passa direttamente alla successiva.

≡ Esempio Pratico

```
for i in range (10):  
    if i == 2:  
        continue  
    print(i)
```

L'output sarà:

```
0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```

In questo caso quando `i` è uguale a 2, il ciclo salta direttamente alla prossima iterazione senza eseguire il `print()` .

Applicazione di Break e Continue con i cicli del While

Anche con il ciclo di `while` è possibile utilizzare `break` e `continue` :

☰ Esempio con Break

```
x=0
while x <10:
    if x==5:
        break
    print(x)
    x += 1
```

L'output sarà:

```
0
1
2
3
4
```

☰ Esempio con Continue

```
x = 0
while x < 5:
    x += 1
    if x \== 3:
        continue
    print(x)
```

Output:

```
1
2
4
5
```

Confronto con gli if statements e for loops

Cicli `for` e confronto con `while`

Come detto prima un ciclo `for` è utilizzato per iterare su una sequenza (liste, stringa, range, intervallo di numeri, etc.).

```
for variabile in sequenza:  
    #blocco di codice
```

```
for i in range (5): #itera su [0, 1, 2, 3, 4]  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

Differenza con il `while`

- `while` :
è utile quando non si conosce in anticipo il numero di iterazioni.
- `for` :
è ideale quando si conosce l'intervallo o la sequenza da iterare.

If Statement

Le strutture condizionali permettono di eseguire diverse istruzioni in base a determinate condizione .

```
if condizione:  
    # Codice da eseguire se la condzione è vera  
elif condizione_2:  
    #Codice da eseguire se l'altra condizione è vera  
else:  
    #Codice da eseguire se nessuna condizione è vera
```

Casi d'esempio:

```
x = int(input("inserisce un numero:"))
if x > 0:
    print("Positivo")
elif x == 0:
    print("Zero")
else:
    print("Negativo")
```

15. Se si inserisce 5 , l'output sarà:

Positivo

16. Se l'utente inserisce 0 , l'output sarà:

Zero

17. Se l'utente inserisce -3 , l'output sarà:

Negativo

☰ Per Ricapitolare

- **Ciclo while :**
utile per eseguire ripetizioni basate su condizioni.
- **Ciclo for :**
ideale per iterare su sequenze.
- **break :**
termina un ciclo .
- **continue :**
salta un'iterazione .
- **Condizioni:**
permettono decisioni logiche nel codice .

Annidare i while

È possibile annidare un ciclo while all'interno di un altro ciclo while per creare iterazioni multiple.

Esempio della tabella dei numeri

```
i = 1
while i <= 3: # Ciclo esterno
    j = 1
    while j <= 3: # Ciclo interno
        print(f"i={i}, j={j}")
        j += 1
    i += 1
```

Output:

```
i=1, j=1
i=1, j=2
i=1, j=3
i=2, j=1
i=2, j=2
i=2, j=3
i=3, j=1
i=3, j=2
i=3, j=3
```

Spiegazione:

18. Ciclo Esterno (i):

- Il ciclo esterno controlla la variabile `i`, che inizia con valore `1` e incrementa di `1` ad ogni iterazione.
- La condizione `i <= 3` mantiene attivo il ciclo fintanto che `i` è minore o uguale a `3`.

19. Ciclo Interno (j):

- Ad ogni iterazione del ciclo esterno, viene eseguito il ciclo interno.
- La variabile `j` viene inizializzata a `1` per ogni iterazione del ciclo esterno e incrementa di `1` finché `j <= 3`.

20. Stampa dei valori:

- Per ogni combinazione di `i` e `j`, il programma stampa i valori correnti delle due variabili.

21. Incrementi:

- Una volta completato il ciclo interno (quando `j` supera `3`), il controllo

ritorna al ciclo esterno, che incrementa `i` e riavvia il ciclo interno con una nuova inizializzazione di `j`.

Iterazioni Ciclo Esterno(<code>i</code>)	Iterazioni ciclo Interno(<code>j</code>)	Output Stampato
<code>i = 1</code>	<code>j = 1, 2, 3</code>	<code>i = 1, j = 1</code>
		<code>i=1, j=2</code>
		<code>i=1, j=3</code>
<code>i = 2</code>	<code>i = 2</code>	<code>i=2, j=1</code>
		<code>i=2, j=2</code>
		<code>i=2, j=3</code>
<code>i = 3</code>	<code>j = 1, 2, 3</code>	<code>i=3, j=1</code>
		<code>i=3, j=2</code>
		<code>i = 3, j = 3</code>

While annidato nei for

Un ciclo di `while` può essere utilizzato all'interno di un ciclo `for`.

Esempio: Contare fino a un limite per ogni elemento di una lista

```
for x in [2, 4, 6]:
    print(f"Tabella di {x}:")
    count = 1
    while count <= 3:
        print(f"{x} x {count} = {x * count}")
        count += 1
```

Output:

```
Tabella di 2:
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
Tabella di 4:
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
Tabella di 6:
```



```
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
```

Spiegazione:

22. Il Ciclo `for` (Iterazione sugli elementi della lista):

- Il ciclo `for` itera sugli elementi della lista `[2, 4, 6]` .
- Ad ogni iterazione, il valore corrente dell'elemento viene assegnato alla variabile `x` .

- Esecuzione del ciclo `for` :
 - **Prima iterazione:** `x = 2`
 - **Seconda iterazione:** `x = 4`
 - **Terza iterazione:** `x = 6`

23. Stampa del messaggio iniziale per ogni valore di `x` :

- Ad ogni iterazione del ciclo `for` , viene stampata una riga che annuncia la "tabella" per il numero `x` .

Esempio:

- Quando `x = 2` , viene stampato: Tabella di 2: .
- Quando `x = 4` , viene stampato: Tabella di 4: .
- Quando `x = 6` , viene stampato: Tabella di 6: .

24. Ciclo `while` Annidato:

- All'interno del ciclo `for` , è presente un ciclo `while` che genera i valori della tabella di moltiplicazione per il numero `x` .

Dettaglio del ciclo `while` :

- La variabile `count` viene inizializzata a `1` all'inizio di ogni iterazione del ciclo `for` .
- Il ciclo `while` esegue iterazioni finché `count <= 3` .
- Ad ogni iterazione:
 - Viene calcolato il prodotto tra `x` e `count` : `x * count` .
 - Viene stampata la riga corrispondente della tabella.
 - La variabile `count` viene incrementata di `1` .

25. Comportamento completo del programma:

- Per ogni valore di `x` nella lista `[2, 4, 6]` , il ciclo `while` calcola e stampa tre righe della tabella di moltiplicazione per quel numero.

Esecuzione Dettagliata

Prima iterazione del ciclo `for` (**x = 2**):

- `print(f"Tabella di 2:")` stampa: Tabella di 2: .
- **Ciclo** `while` :
 - `count = 1` : Stampa `2 x 1 = 2` .
 - `count = 2` : Stampa `2 x 2 = 4` .
 - `count = 3` : Stampa `2 x 3 = 6` .

Seconda iterazione del ciclo `for` (**x = 4**):

- `print(f"Tabella di 4:")` stampa: Tabella di 4: .
- **Ciclo** `while` :
 - `count = 1` : Stampa `4 x 1 = 4` .
 - `count = 2` : Stampa `4 x 2 = 8` .
 - `count = 3` : Stampa `4 x 3 = 12` .

Terza iterazione del ciclo `for` (**x = 6**):

- `print(f"Tabella di 6:")` stampa: Tabella di 6: .
- **Ciclo** `while` :
 - `count = 1` : Stampa `6 x 1 = 6` .
 - `count = 2` : Stampa `6 x 2 = 12` .
 - `count = 3` : Stampa `6 x 3 = 18` .

If annidato in un while

Un'istruzione condizionale `if` può essere usata all'interno di un ciclo

`while` per controllare condizioni specifiche.

Esempio: Sommare solo i numeri pari fino a un limite:

```
limite = 10
x = 1
somma = 0
while x <= limite:
    if x % 2 == 0: # Verifica se il numero è pari
        somma += x
    x += 1
print(f"La somma dei numeri pari da 1 a 10 è {somma}.")
```

Output:

La somma dei numeri pari da 1 a 10 è 30.

Spiegazione dettagliata

1. Inizializzazione delle variabili:

- `limite`:
Imposta il valore massimo fino a cui iterare (incluso). In questo caso, il `limite` è 10.
- `x`:
È inizializzato a 1 ed è il contatore usato per iterare nel ciclo `while`.
- `somma`:
Parte da 0 ed è la variabile che accumula la somma dei numeri pari.

2. Ciclo `while`:

Il ciclo `while` esegue iterazioni finché la condizione `x <= limite` è vera:

- Quando `x` supera il valore di `limite` (cioè 10), il ciclo si interrompe.

3. Condizione `if` annidata:

All'interno del ciclo `while`, c'è una `condizione if` che controlla se il `numero corrente ( x )` è pari:

- La condizione `x % 2 == 0` verifica se il resto della divisione di `x` per 2 è uguale a 0.
 - Se vero, `x` è pari e viene aggiunto alla variabile `somma`.
 - Se falso, non accade nulla, e il ciclo passa all'iterazione successiva.

4. Incremento di `x`

Alla fine di ogni iterazione del ciclo `while`, il valore di `x` viene incrementato di 1 con `x += 1` per passare al numero successivo.

5. Stampa del risultato

Dopo che il ciclo si è concluso, il programma stampa il valore accumulato in `somma`, mostrando la somma di tutti i numeri pari fino al limite specificato.

Esecuzione dettagliata

Vediamo passo passo cosa succede durante l'esecuzione:

Iterazione	x	$x \% 2 \neq 0$	Azione	Risultato
1	2	False	x è dispari, salta l'if	0
2	2	True	aggiunge 2 a somma	2
3	3	False	x è dispari, salta if	2
4	4	True	aggiunge 4 a somma	6
5	5	False	x è dispari, salta l'if	6
6	6	True	Aggiunge 6 a somma	12
7	7	False	x è dispari, salta il if	12
8	8	True	Aggiunge 8 a somma	20
9	9	False	x è dispari, salta il if	20
10	10	True	Aggiunge 10 a somma	30
Fine	11	-	il ciclo si interrompe	30

🔑 Concetti chiave

- **Ciclo while :**
Usato per iterare fino a un certo limite.
- **Condizione if :**
Annidata per filtrare solo i numeri pari .
- **Operatore % :**
Il modulo % è usato per verificare se un numero è pari o dispari.
- **Accumulatore (somma):**
Una variabile che raccoglie valori incrementali.

Combinazioni complesse

Si possono creare annidazioni complesse: ad esempio un ciclo di `for` dentro un `while` e includere condizioni `if` .

Esempio: Stampare i valori di una matrice

```
matrice = [
    [1, 2, 3],
    [4, 5, 6],
```

```

    [7, 8, 9]
]

riga = 0
while riga < len(matrice):
    for colonna in matrice[riga]:
        if colonna % 2 == 0:
            print(f"Numero pari: {colonna}")
    riga += 1

```

Output:

```

Numero pari: 2
Numero pari: 4
Numero pari: 6
Numero pari: 8

```

🕒 Linee guida logiche nel uso combinato di for, while, if, elif, else, break, e continue

Non esiste una gerarchia rigida di utilizzo o obbligatoria nell'uso combinato di for, while, if, elif, else, break, e continue.

Tuttavia ci sono delle **linee guida logiche** e delle buone pratiche che determinano come strutturare il codice per ottenere il comportamento desiderato.

1. Gerarchia Logica

La combinazione di questi costrutti dipende dalla logica del problema che si sta risolvendo.

I cicli (for e while):

- I cicli sono usati per iterare su una sequenza (for) o per ripetere un'azione fino a una condizione (while).
- All'interno dei cicli puoi usare condizioni (if, elif, else) per decidere cosa fare in base a determinati criteri.

Le condizioni (if, elif, else):

- Queste sono usate per il controllo del flusso logico.
- Possono essere annidate all'interno dei cicli per verificare criteri specifici in ogni iterazione.

break e continue:

- break:

Uscire immediatamente dal ciclo in cui si trova.

- `continue` :

Saltare l'iterazione corrente e passare alla successiva.

- Si usano **all'interno di cicli**, spesso combinati con condizioni (`if`), per modificare il flusso.

2. Linee guida pratiche

Quando usare i cicli:

- Usa un `for` quando sai in anticipo quante iterazioni eseguire (es.: iterare su una lista, su un range di numeri).
- Usa un `while` quando la condizione di uscita non è determinata a priori o dipende da una variabile che cambia durante l'esecuzione.

Condizioni (`if` , `elif` , `else`):

- Puoi annidarle ovunque sia necessario, ma evita annidamenti troppo profondi per mantenere il codice leggibile.
- Ogni ciclo può contenere condizioni per verificare criteri specifici.

`break` e `continue` :

- `break` :
Usalo quando sai che non c'è più bisogno di continuare l'iterazione (es.: hai trovato il valore cercato in un ciclo di ricerca).
- `continue` :
Usalo per saltare iterazioni inutili (es.: evitare calcoli per numeri dispari in un ciclo che processa solo numeri pari).

Tips & Ticks ✓

Una gerarchia tipo in uno scenario complesso potrebbe essere:

- Un **ciclo esterno** (`for` o `while`) per gestire le iterazioni principali.
- All'interno, una o più condizioni (`if` , `elif` , `else`) per il controllo logico.
- Opzionalmente, `break` o `continue` per gestire casi particolari.
- Possibili cicli annidati se il problema richiede operazioni gerarchiche.

☰ **Trovare e sommare i numeri pari in una sequenza fino a un limite** >

```

limite = 20
numeri = [3, 7, 8, 12, 15, 18, 21, 24]
somma = 0

for numero in numeri:
    if numero > limite: # Se il numero supera il
        limite, interrompi il ciclo
        break
    if numero % 2 != 0: # Se il numero è dispari,
        salta questa iterazione
        continue
    somma += numero # Aggiungi il numero pari alla
        somma

print(f"La somma dei numeri pari sotto {limite} è
{somma}")

```

Esecuzione:

1. Il ciclo `for` itera su ogni elemento della lista `numeri`.
2. La condizione `if numero > limite` interrompe il ciclo se un numero supera il limite.
3. La condizione `if numero % 2 != 0` salta i numeri dispari.
4. Se entrambe le condizioni sono false, il numero viene aggiunto alla somma.

✓ Cosa evitare

- **Annidamenti eccessivi:** Troppi cicli e condizioni annidati rendono il codice difficile da leggere. Se necessario, usa funzioni per semplificare.
- **Abuso di `break` e `continue`:** Usali solo se migliorano la chiarezza logica del codice.
- **Cicli infiniti non controllati:** Quando usi `while`, assicurati che la condizione diventi falsa a un certo punto.

Loop con un blocco `else`

Un modello raramente usato disponibile in Python è l'istruzione `else` come parte di un ciclo `for` o `while`.

Abbiamo discusso in precedenza del blocco `else`:

viene eseguito se tutte le istruzioni `if` ed `elif` restituiscono `False`.

Il loop-`else` è forse una delle affermazioni con nomi più confusi in Python:

l'`else` viene eseguito **solo se il ciclo termina senza eseguire un'istruzione `break`**, ovvero quando nessuna condizione all'interno del ciclo ha richiesto l'interruzione anticipata.

Quindi puoi pensarlo come un'istruzione `nobreak`:

cioè, **il blocco `else` viene eseguito solo se il ciclo termina naturalmente, senza incontrare un'istruzione `break`.**

Un esempio classico è l'algoritmo del Crivello di Eratostene per trovare numeri primi, nel codice seguente, il ciclo `else` viene eseguito solo quando il numero non è divisibile per nessun elemento già presente nella lista dei numeri primi trovati, indicando che il numero stesso è primo.

```
L = [] # Lista vuota che conterrà i numeri primi trovati
nmax = 30 # Limite massimo fino a cui cercare i numeri primi

for n in range(2, nmax): # Itera sui numeri da 2 a 29
    for factor in L: # Controlla i numeri già trovati (che sono
        primi)
        if n % factor == 0: # Se n è divisibile per un numero primo
            già trovato
            break # Esce dal ciclo interno, quindi n NON è primo
        else: # Eseguito solo se il ciclo interno termina senza trovare
            un divisore
            L.append(n) # Aggiunge il numero alla lista dei numeri
            primi
print(L) # Stampa la lista dei numeri primi trovati fino a nmax
```

Output:

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```


L'istruzione `else` viene eseguita solo se nessuno dei fattori divide il numero dato.
L'istruzione `else` funziona in modo simile con il ciclo `while`.

Come funziona il ciclo `for...else` in questo contesto?

26. **Ciclo esterno** (`for n in range(2, nmax)`):

- Scorre tutti i numeri da `2` fino a `nmax - 1`.

27. **Ciclo interno** (`for factor in L`):

- Controlla tutti i numeri primi trovati finora (contenuti nella lista `L`).
- Se il numero `n` è divisibile per uno di questi numeri (`n % factor == 0`), allora **non è primo**.
- In questo caso, viene eseguita l'istruzione `break`, interrompendo il ciclo interno.

28. **Blocco** `else`:

- L'`else` si esegue **solo** se il ciclo interno termina **senza eseguire il** `break`, cioè quando **nessun numero ha diviso** `n`.
- Questo significa che `n` **è primo** e quindi viene aggiunto alla lista `L`.

Esempio passo-passo con `nmax = 10`:

- `n = 2`:
 - Lista `L` è vuota → Nessun ciclo interno → **Else eseguito** → Aggiungi `2`.
- `n = 3`:
 - Controlla se è divisibile per `2` (non lo è) → **Else eseguito** → Aggiungi `3`.
- `n = 4`:
 - Controlla se è divisibile per `2` → **Sì**, esegue `break`.
- `n = 5`:
 - Controlla `2`, `3` → Nessun divisore → **Else eseguito** → Aggiungi `5`.
- `n = 6`:
 - Divisibile per `2` → `break`.
- `n = 7`:
 - Controlla `2`, `3` → Nessun divisore → **Else eseguito** → Aggiungi `7`.